ALLUXIO

Alluxio 数据分布策略

张策 / *Alluxio Contributor*

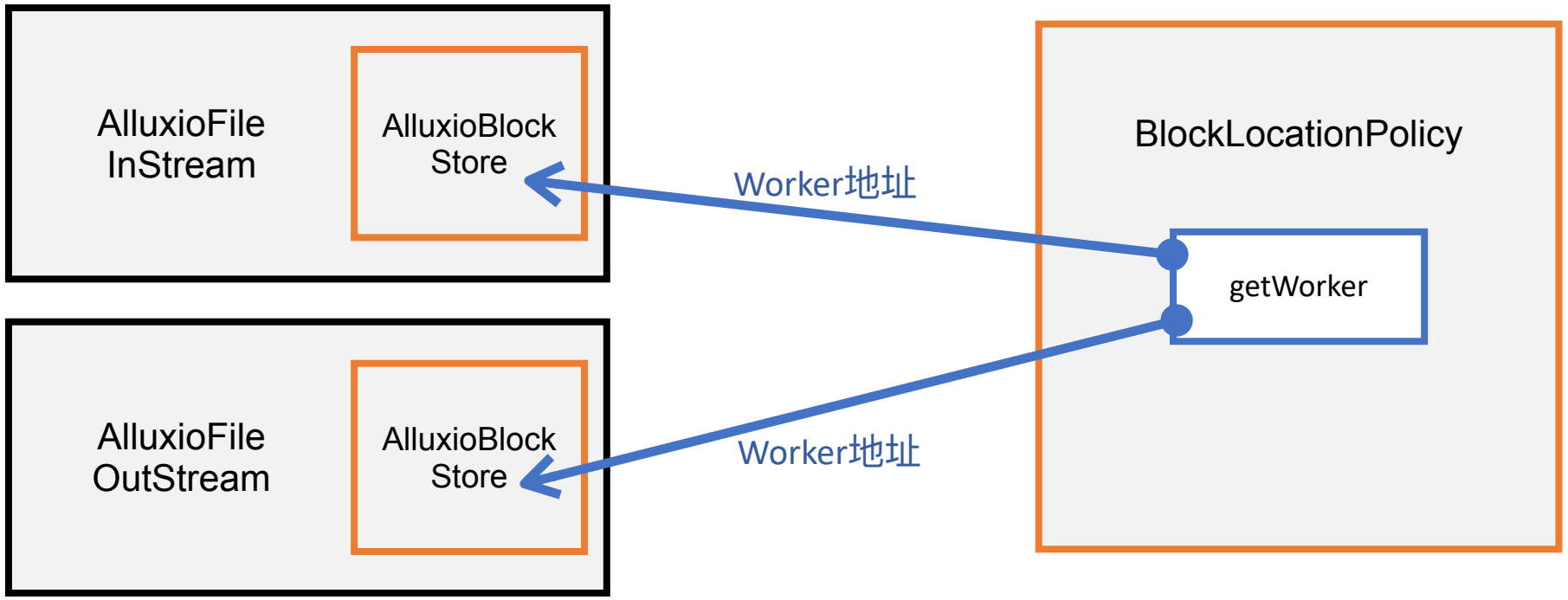
ALLUXIO OPEN SOURCE COMMUNITY

2020.04

目录

- 工作原理
- 常用数据分布策略
- 自定义策略

1.工作原理


AlluxioFile
InStream
AlluxioBlock
Store
AlluxioFile
OutStream
AlluxioBlock
Store
Worker地址
Worker地址
BlockLocationPolicy
getWorker

1.工作原理

alluxio.user.ufs.block.read.location.policy

alluxio.user.block.read.location.policy.class



```
./bin/spark-submit.sh \
—conf spark.executor.extraJavaOptions= \
“-Dalluxio.user.ufs.block.read.location.policy \
 =alluxio.client.block.policy.LocalFirstPolicy” \
…
```

2.常用数据分布策略

- LocalFirstPolicy - 本地优先策略
- LocalFirstAvoidEvictionPolicy - 预留内存的本地优先策略
- MostAvailableFirstPolicy - 剩余容量最大优先策略
- RoundRobinPolicy - 轮询策略
- DeterministicHashPolicy - 限制主机数目策略
- SpecificHostPolicy - 指定HOST策略

2.1本地优先策略

LocalFirstPolicy

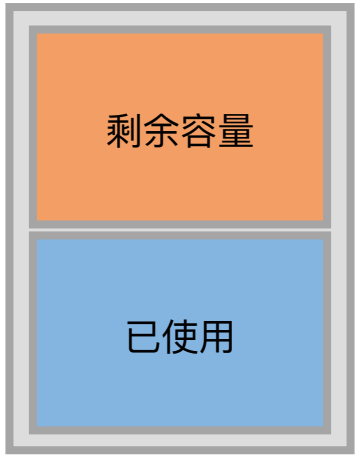
- 默认策略
- 优先返回本地worker
- 如果本地worker没有足够block，随机选择一个活跃的worker

LocalFirstAvoidEvictionPolicy

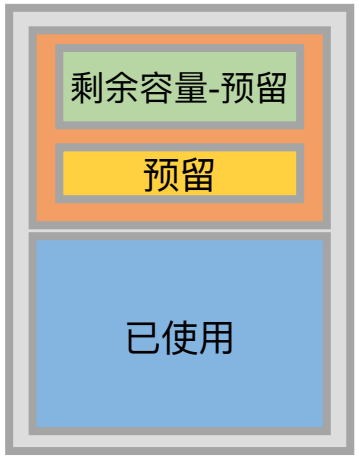
- 具备预留空间功能的升级版LocalFirstPolicy，没有满足条件的worker则退回LocalFirstPolicy
- 配置 **alluxio.user.block.avoid.eviction.policy.reserved.size.bytes** 参数设置worker预留的内存大小

2.1本地优先策略

LocalFirstPolicy



LocalFirstAvoidEvictionPolicy



2.1本地优先策略

LocalFirstPolicy

- 优点
  - A. 当空间足够、高并发读入Alluxio内存数据时读取、写入性能好
- 缺点
  - A. 集群容易发生数据倾斜
  - B. 高并发写入worker有可能挤垮内存
  - C. 计算任务有很多task同时读相同的文件块时，导致副本数增加；读取底层存储相同数据块导致速度变慢

LocalFirstAvoidEvictionPolicy

- 优点
  - A. 由于预留空间的保护，降低了worker内存被挤垮的风险
- 缺点
  - A. 同LocalFirstPolicy的缺点A和缺点C

2.2负载均衡策略

MostAvailableFirstPolicy

- 选择剩余容量最大的一个Worker

RoundRobinPolicy

- 在剩余容量足够的Worker中轮流写入

2.2负载均衡策略

MostAvailableFirstPolicy

- 优点
  - A. 能够最大程度上利用集群空间，应用在异构集群能够充分利用内存
  - B. 负载均衡
- 缺点
  - A. 存在因信息更新不及时导致多个客户端集中写入同一个worker的风险

RoundRobinPolicy

- 优点
  - A. 能够避免MostAvailableFirstPolicy的缺点
  - B. 负载均衡
- 缺点
  - A. 无法最高效利用内存
  - B. 集群异构时无法根据worker的内存总量实现物尽其用

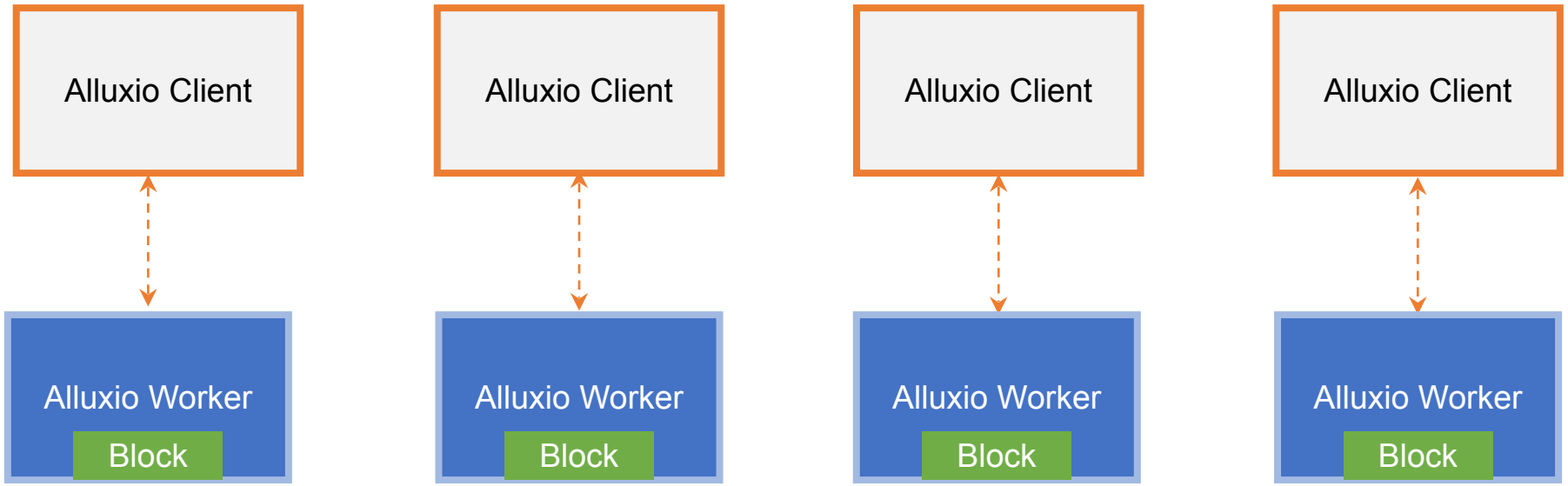
2.3限制主机数目策略

DeterministicHashPolicy

- 在高并发访问UFS的场景下能够有效限制文件副本数
- 配置**alluxio.user.ufs.block.read.location.policy.deterministic.hash.shards**参数设置加载数据的worker数目
- 不同的Alluxio客户端会使用同样的哈希函数来决定去哪几个worker去读并缓存所需要的文件，集群添加、移除worker时，计算出的worker会随之变化

2.3限制主机数目策略

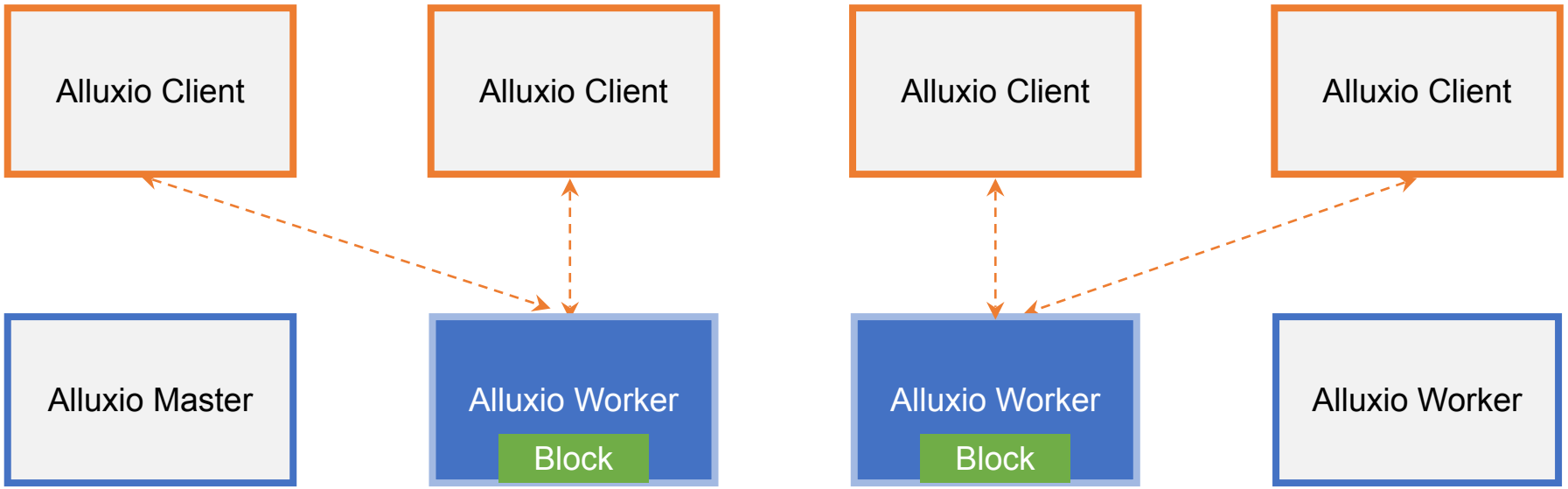
DeterministicHashPolicy-使用前



4副本

2.3限制主机数目策略

DeterministicHashPolicy-使用后



2副本

3.自定义策略

- 实现alluxio.client.block.policy.BlockLocationPolicy接口
- 默认策略需要拥有一个包含alluxio.conf.AlluxioConfiguration参数的构造函数
- 使用异步写（ASYNC_THROUGH）时文件的所有block需要写入相同worker

要点回顾

- Alluxio Client 通过**BlockLocationPolicy**接口实现类的**getWorker**方法决定去哪个Worker读写数据
- 常用策略
  A. 本地优先策略。**LocalFirstAvoidEvictionPolicy**能够降低内存挤爆的风险
  B. 负载均衡策略。**MostAvailableFirstPolicy**内存利用率高，**RoundRobinPolicy**更加安全
  C. 限制主机数目策略。**DeterministicHashPolicy**提高第一次高并发读取底层存储文件块的性能，有效控制第一次加载时的文件副本数
- 使用自定义策略满足个性化需求

Join Our User Community



Join Slack channel
alluxio.io/slack



Join meetup groups near you
alluxio-open-source-community/

Wechat Public Account